

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

BÁO CÁO GIỮA KỲ

Học phần ET4710 – Lập trình ứng dụng di động

ROOMHUB
ỨNG DỤNG ĐẶT PHÒNG HỌC VÀ PHÒNG HỌP

Nhóm thực hiện:	Hoàng Văn Tùng	20252389M
	Đỗ Thùy Linh	20252478M
	Nguyễn Đình Phúc	20251012M
	Nguyễn Việt Dũng	20252690M

Giảng viên hướng dẫn: PGS.TS. Đỗ Trọng Tuấn

<https://github.com/xwomen1/classroom-booking>

HÀ NỘI, THÁNG 10 NĂM 2026

TÓM TẮT

RoomHub là ứng dụng Android hỗ trợ quản lý vòng đời mượn phòng học và phòng họp, từ tìm phòng, gửi yêu cầu, phê duyệt, bàn giao quyền truy cập đến check-in, check-out và xử lý bảo trì. Ứng dụng phục vụ hai vai trò: giảng viên và cán bộ quản lý cơ sở vật chất. Dữ liệu có thể hoạt động hoàn toàn cục bộ trên một thiết bị; dịch vụ Java Spring Boot cung cấp nền tảng đồng bộ khi chế độ trực tuyến được kích hoạt. Với phòng dùng khóa số, hệ thống tạo mã PIN tạm thời theo lịch đặt và giao tiếp với nền tảng OneIoT.

Báo cáo trình bày phạm vi MVP, kiến trúc và tổ chức mã nguồn theo module, các quy tắc nghiệp vụ chính, kết quả giao diện và quy trình kiểm thử. Phiên bản hiện tại đã vượt qua 36 ca kiểm thử Jest, kiểm tra kiểu TypeScript, ESLint, kiểm thử tích hợp với Spring Boot/H2 và kiểm tra cài đặt bản Release trên điện thoại Android. Chế độ cục bộ được chọn cho bản trình diễn để ứng dụng hoạt động ổn định mà không phụ thuộc máy chủ.

MỤC LỤC

Tóm tắt	i
Danh mục thuật ngữ và từ viết tắt	vi
1 GIỚI THIỆU	1
1.1 Bối cảnh và bài toán	1
1.2 Mục tiêu	1
1.3 Phạm vi	1
1.4 Định hướng giải pháp và cấu trúc báo cáo	2
2 TRIỂN KHAI HỆ THỐNG	3
2.1 Tổng quan hệ thống	3
2.2 Phạm vi MVP	4
2.3 Tổ chức mã nguồn ứng dụng	6
2.4 Lập trình theo từng module	6
2.4.1 Module auth và account_management	7
2.4.2 Module room_management	7
2.4.3 Module booking	8
2.4.4 Module schedule_maintenance	8
2.4.5 Module access_control	9
2.4.6 Module smart_lock và tích hợp OneIoT	9
2.4.7 Module notifications, profile và configuration	9
2.4.8 Booking server	10
2.5 Điều hướng giao diện	10
3 KIỂM THỬ VÀ ĐÁNH GIÁ	12
3.1 Mục tiêu và phạm vi kiểm thử	12
3.2 Môi trường kiểm thử	12
3.3 Quy trình kiểm thử	12
3.4 Nhóm ca kiểm thử chính	13
3.5 Kết quả	14
3.6 Giới hạn và hướng kiểm chứng tiếp theo	15
Kết luận	16
Tài liệu tham khảo	17

A	THÀNH PHẦN VÀ CẤU TRÚC DỰ ÁN	18
A.1	Cấu trúc kho mã nguồn	18
A.2	Build và cài đặt	18
A.3	Tổng hợp tài liệu và phụ lục	19
A.4	Phân công nhóm	19
B	PROMPT HỖ TRỢ QUÁ TRÌNH PHÁT TRIỂN	20
B.1	Nguyên tắc sử dụng	20
B.2	Các nhóm prompt tiêu biểu	20
B.3	Quy trình kiểm soát đầu ra	21
C	ẢNH GIAO DIỆN ỨNG DỤNG HIỆN TẠI	22

DANH MỤC HÌNH VẼ

2.1	Kiến trúc tổng thể của hệ thống RoomHub	4
2.2	Vòng đời trạng thái của một booking	5
2.3	Luồng điều hướng giao diện theo hai vai trò	11
3.1	Quy trình kiểm thử và xác nhận bản Release	13
C.1	Màn hình chính theo hai vai trò của ứng dụng hiện tại	22
C.2	Tìm phòng và tạo yêu cầu trên cùng một màn hình	23
C.3	Hai cách xem phòng theo tầng ở ứng dụng hiện tại	24
C.4	Giao diện quản lý phòng và cấu hình nghiệp vụ	25

DANH MỤC BẢNG BIỂU

2.1	Các thành phần chính của hệ thống RoomHub	3
2.2	Phạm vi MVP đã chốt	5
2.3	Danh sách module của ứng dụng	7
3.1	Môi trường kiểm thử của phiên bản báo cáo	12
3.2	Các nhóm ca kiểm thử chính	13
3.3	Kết quả kiểm thử gần nhất	15
A.1	Tài liệu hỗ trợ cho báo cáo	19
A.2	Phân công công việc trong nhóm	19
B.1	Prompt tiêu biểu trong quá trình phát triển	20

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
APK	Gói cài đặt ứng dụng Android (Android Package)
Booking	Yêu cầu đặt phòng gồm phòng, ngày, khung giờ và mục đích sử dụng
CRUD	Các thao tác tạo, đọc, cập nhật và xóa dữ liệu
IoT	Internet vạn vật (Internet of Things)
MQTT	Giao thức truyền thông điệp theo mô hình publish/subscribe
MVP	Sản phẩm khả dụng tối thiểu (Minimum Viable Product)
No-show	Trường hợp người đặt không đến sử dụng phòng đã được duyệt
PIN	Mã số dùng để mở khóa (Personal Identification Number)
TLS	Giao thức bảo mật tầng vận chuyển (Transport Layer Security)
UAT	Kiểm thử chấp nhận người dùng (User Acceptance Testing)

CHƯƠNG 1

GIỚI THIỆU

1.1 Bối cảnh và bài toán

Nhu cầu sử dụng phòng học và phòng họp của giảng viên diễn ra thường xuyên, trong khi quy trình đăng ký tại nhiều đơn vị vẫn phụ thuộc vào trao đổi trực tiếp với cán bộ quản lý cơ sở vật chất. Người mượn khó tra cứu nhanh phòng còn trống theo thời gian, sức chứa và thiết bị. Người quản lý cũng phải theo dõi thủ công lịch sử dụng, lịch bảo trì, việc bàn giao chìa khóa hoặc thẻ từ và các trường hợp người đặt không đến.

Các hạn chế trên dễ dẫn đến trùng lịch, lựa chọn phòng không phù hợp và lãng phí tài nguyên. Khi số lượng phòng và lượt đăng ký tăng, việc kiểm tra bằng bảng tính hoặc tin nhắn riêng lẻ không còn bảo đảm tính nhất quán của dữ liệu.

1.2 Mục tiêu

Dự án xây dựng ứng dụng di động **RoomHub** nhằm quản lý xuyên suốt một lượt mượn phòng, từ tìm kiếm, gửi yêu cầu, phê duyệt đến bàn giao quyền truy cập, sử dụng và kết thúc phiên. Hệ thống hướng tới các mục tiêu sau:

- giúp giảng viên tìm và đặt phòng theo tầng, thời gian, sức chứa và thiết bị;
- hỗ trợ cán bộ quản lý phê duyệt, đổi phòng, quản lý phòng và lịch bảo trì;
- ngăn các trường hợp trùng lịch, vượt giới hạn đặt phòng hoặc đặt vào thời gian bảo trì;
- hỗ trợ cả phòng dùng khóa cơ/thẻ từ và phòng dùng khóa mã số;
- lưu vết các sự kiện quan trọng như phê duyệt, cấp mã, check-in, check-out và no-show.

1.3 Phạm vi

RoomHub phục vụ hai vai trò. **Quản trị viên** là cán bộ quản lý cơ sở vật chất, có quyền quản lý tài khoản, phòng, yêu cầu đặt phòng, lịch bảo trì, khóa thông minh và các tham số nghiệp vụ. **Người dùng** là giảng viên, có quyền tìm phòng, tạo và quản lý yêu cầu của mình, thỏa thuận thời gian nhận khóa, tạo mã tạm thời khi được cấp quyền và báo sự cố trong thời gian sử dụng phòng.

Mô hình dữ liệu thử nghiệm gồm tám tầng, mỗi tầng bảy phòng. Mỗi phòng có sức chứa, danh sách thiết bị, trạng thái và một trong hai loại khóa: khóa cơ/thẻ từ hoặc khóa mã số. Với khóa mã số, ứng dụng có thể gửi lệnh tạo PIN tạm thời tới nền tảng OneIoT theo thời gian của lịch đặt.

1.4 Định hướng giải pháp và cấu trúc báo cáo

Ứng dụng Android được phát triển bằng React Native và TypeScript [1]. Ở chế độ cục bộ, dữ liệu được lưu bằng Async Storage [2], cho phép bản Release hoạt động độc lập trên một điện thoại. Dịch vụ Java Spring Boot [3] cung cấp REST API và cơ sở dữ liệu H2 để kiểm thử cơ chế đồng bộ. Chế độ dữ liệu được lựa chọn thông qua tham số `ENABLE_REMOTE_SYNC`; bản trình diễn trong báo cáo sử dụng chế độ cục bộ để không phụ thuộc trạng thái của máy chủ.

Đối với phòng dùng khóa mã số, ứng dụng giao tiếp với OneIoT qua MQTT trên TLS [4] và định dạng bản tin oneM2M [5]. Mã PIN được tạo và lưu theo lịch đặt trước khi gửi; khi chưa có kết nối, lệnh được giữ ở trạng thái chờ và được gửi khi phiên kết nối hợp lệ được thiết lập.

Chương 2 trình bày phạm vi MVP, kiến trúc và tổ chức mã nguồn. Chương 3 mô tả phương pháp kiểm thử và kết quả của phiên bản hiện tại. Phụ lục A chỉ ra vị trí các thành phần chính trong kho mã nguồn; Phụ lục B tổng hợp cách nhóm sử dụng trợ lý lập trình và kiểm soát đầu ra.

CHƯƠNG 2

TRIỂN KHAI HỆ THỐNG

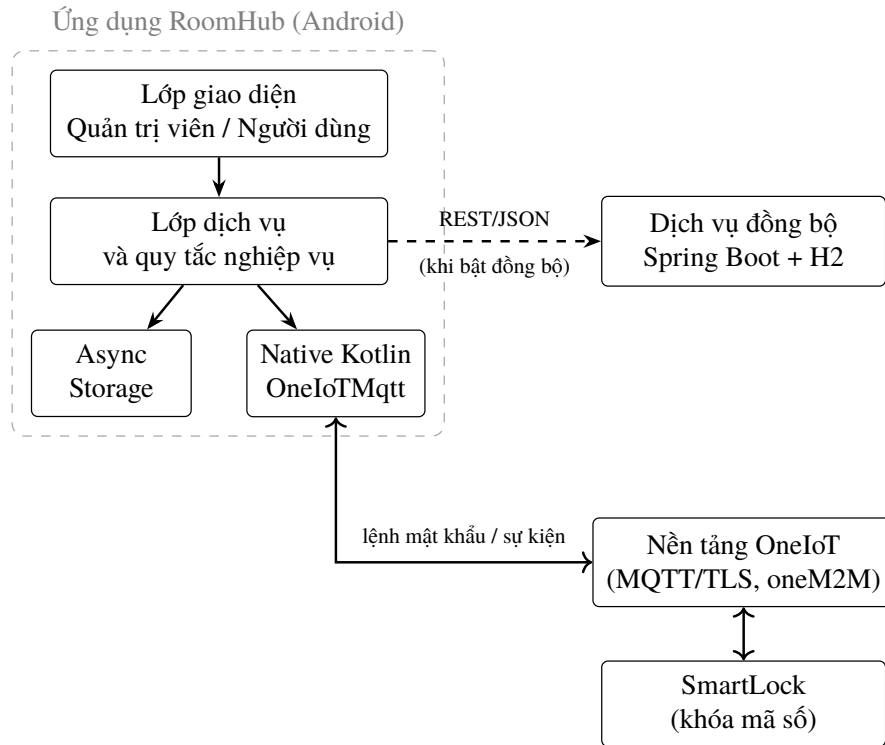
2.1 Tổng quan hệ thống

Hệ thống RoomHub gồm ứng dụng di động, dịch vụ đồng bộ và hai công cụ hỗ trợ tích hợp khóa thông minh. Bảng 2.1 nêu vai trò của từng thành phần; cấu trúc thư mục chi tiết được trình bày tại Phụ lục A.

Bảng 2.1. Các thành phần chính của hệ thống RoomHub

Thành phần	Công nghệ	Vai trò
Ứng dụng di động	React Native, TypeScript, Kotlin	Giao diện cho hai vai trò, nghiệp vụ đặt phòng, lưu dữ liệu cục bộ và giao tiếp OneIoT qua module native.
Dịch vụ đồng bộ	Java, Spring Boot, H2	REST API cho tài khoản, phòng, lịch đặt, bảo trì, quyền truy cập và thông báo.
Mô phỏng SmartLock	Python	Mô phỏng khóa để kiểm tra bản tin tạo mật khẩu và sự kiện truy cập.
Công cụ giao tiếp	Python, MQTT	Tham chiếu cơ chế kết nối và định dạng bản tin trao đổi với OneIoT.

Hình 2.1 mô tả kiến trúc tổng thể. Ứng dụng được tổ chức theo ba lớp: lớp màn hình (*screens*), lớp dịch vụ nghiệp vụ (*services*) và lớp lưu trữ.



Hình 2.1. Kiến trúc tổng thể của hệ thống RoomHub

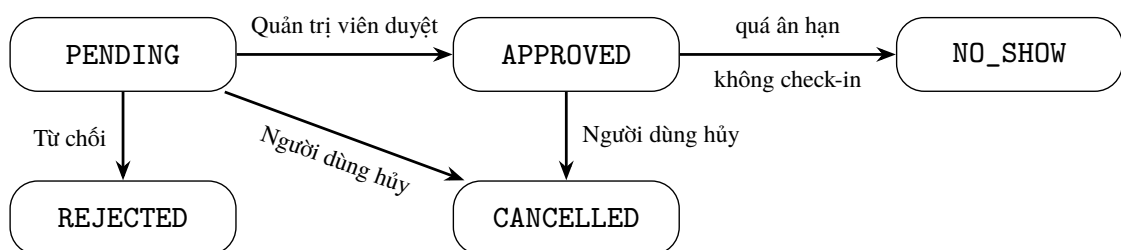
2.2 Phạm vi MVP

Tiêu chí hoàn thành MVP gồm: (i) luồng nghiệp vụ chính chạy được trên thiết bị thật, (ii) bản cài đặt Release tái lập được từ mã nguồn, (iii) các quy tắc nghiệp vụ quan trọng có kiểm thử tự động, và (iv) không còn lỗi nghiêm trọng đã biết trong luồng chính. Bảng 2.2 phân định rõ phần thuộc MVP và phần để lại cho giai đoạn sau.

Bảng 2.2. Phạm vi MVP đã chốt

Nhóm chức năng	Nội dung	Trạng thái
Tài khoản	Đăng nhập, đăng ký, đổi/khôi phục mật khẩu bằng mã khôi phục, hồ sơ, điều hướng theo vai trò; quản trị viên quản lý tài khoản.	Đã triển khai
Tìm phòng	Lọc theo tầng, ngày giờ, sức chứa, thiết bị, loại khóa; sơ đồ tầng với màu trạng thái.	Đã triển khai
Đặt và duyệt	Tạo yêu cầu, kiểm tra quy tắc; quản trị viên duyệt, từ chối hoặc đổi phòng; người dùng hủy theo hạn.	Đã triển khai
Đặt lặp tuần	Tạo chuỗi tối đa 8 tuần, sửa/hủy các lượt tương lai, duyệt cả chuỗi.	Đã triển khai
Phòng và bảo trì	Quản lý phòng, lịch bảo trì, yêu cầu bảo trì từ người dùng và lịch sử sử dụng.	Đã triển khai
Truy cập phòng	Khóa cơ/thẻ từ: thỏa thuận lịch nhận khóa, ủy quyền nhận hộ. Khóa số: cấp quyền, tạo PIN 7 chữ số, gửi lệnh qua OneIoT.	Đã triển khai
Điểm danh	Ghi nhận check-in/check-out và no-show sau thời gian ân hạn.	Đã triển khai
Thông báo, cấu hình	Thông báo theo sự kiện, thông báo nổi ưu tiên; quản trị viên cấu hình quy tắc.	Đã triển khai
Đồng bộ nhiều thiết bị	Dùng booking server làm nguồn dữ liệu chung; đã kiểm thử với máy chủ H2 cục bộ.	Đang hoàn thiện
Kiểm chứng khóa thật	Xác nhận SmartLock thật nhận và áp dụng mật khẩu.	Giai đoạn sau
Mở rộng	QR check-in, thống kê mức sử dụng, gợi ý phòng nâng cao.	Giai đoạn sau

Các quy tắc nghiệp vụ của MVP được tham số hóa trong kiểu `AppConfiguration` để quản trị viên thay đổi ngay trong ứng dụng. Cấu hình mặc định yêu cầu đặt trước từ 1 đến 3 ngày; khi cần cho phép đặt cùng ngày, quản trị viên có thể đặt giá trị tối thiểu về 0. Mỗi người dùng có tối đa 2 yêu cầu đang chờ hoặc sắp dùng; hạn hủy và hạn đổi phòng là 30 phút trước giờ bắt đầu; mã PIN có hiệu lực từ 10 phút trước đến 10 phút sau phiên; thời gian ân hạn trước khi ghi nhận no-show là 15 phút. Vòng đời trạng thái của một booking được thể hiện ở Hình 2.2.

**Hình 2.2.** Vòng đời trạng thái của một booking

2.3 Tổ chức mã nguồn ứng dụng

Mã nguồn ứng dụng được chia thành ba vùng. Vùng `src/core` chứa thành phần hạ tầng dùng chung: kiểu người dùng, lớp bọc Async Storage, tham số `ENABLE_REMOTE_SYNC` và hàm `apiRequest` gắn token phiên. Vùng `src/shared` chứa các thành phần giao diện dùng chung như `ScreenHeader` và `PasswordInput`. Vùng `src/modules` chứa mười module nghiệp vụ; mỗi module sử dụng cấu trúc `screens/`, `components/`, `services/`, `model/` và công khai API qua `index.ts`. Quy ước này giúp các thành viên phát triển độc lập và giảm xung đột khi hợp nhất mã. Đoạn mã 2.1 minh họa cách lớp dịch vụ lựa chọn nguồn dữ liệu.

Đoạn mã 2.1. Hàm dịch vụ lựa chọn nguồn dữ liệu (rút gọn)

```
export async function createBooking(input: CreateBookingInput) {
  if (isRemoteApiEnabled()) { // chế độ đồng bộ qua server
    const r = await apiRequest<{booking: Booking}>('/api/bookings',
      {method: 'POST', body: input});
    return r.booking;
  }
  await assertAccountRole(input.requesterUsername, 'user');
  const bookings = await getBookings(); // chế độ local: Async Storage
  ...
}
```

2.4 Lập trình theo từng module

Bảng 2.3 tổng hợp mười module, khóa lưu trữ và người phụ trách chính. Các mục con sau đó phân tích những điểm lập trình quan trọng của từng module.

Bảng 2.3. Danh sách module của ứng dụng

Module	Phạm vi	Dữ liệu cục bộ
auth	Đăng nhập, đăng ký, đổi/khôi phục mật khẩu	auth.accounts
account_management	Quản lý và phân quyền tài khoản	dùng auth
room_management	CRUD phòng, tìm phòng, sơ đồ tầng	rooms.records
booking	Đặt, duyệt, hủy, lặp tuần, đổi phòng, điểm danh	booking.records
schedule_maintenance	Lịch và yêu cầu bảo trì	maintenance.*
access_control	Quyền tạo mã, PIN tạm thời	access.roomPinPermissions
smart_lock	Gắn khóa vào phòng, OneIoT, sự kiện khóa	smartLock.primary
notifications	Thông báo và thông báo nổi ưu tiên	theo người dùng
profile	Hồ sơ cá nhân	theo người dùng
configuration	Tham số quy tắc nghiệp vụ	configuration.app

2.4.1 Module auth và account_management

Module auth lưu tài khoản dưới kiểu `AccountRecord`, gồm tên đăng nhập, mật khẩu, mã khôi phục tùy chọn, vai trò và trạng thái hoạt động. Lần chạy đầu, hệ thống khởi tạo hai tài khoản thử nghiệm cho hai vai trò. Khi đăng ký, tên đăng nhập, mật khẩu và xác nhận mật khẩu là các trường bắt buộc; thông tin cá nhân và mã khôi phục được đặt ở nhóm tùy chọn. Hàm `registerAccount` kiểm tra định dạng tên, độ dài mật khẩu và tên không trùng. Hàm `resetPasswordWithRecoveryCode` chỉ hoạt động với tài khoản đã thiết lập mã khôi phục. Các thao tác nghiệp vụ quan trọng gọi `assertAccountRole` để xác nhận tài khoản còn hoạt động và có đúng vai trò. Trong `account_management`, quản trị viên không thể tự khóa, tự đổi vai trò hoặc xóa tài khoản còn lịch đặt đang hoạt động.

2.4.2 Module room_management

Dữ liệu khởi tạo gồm 56 phòng, bảy phòng trên mỗi tầng của tòa nhà tám tầng, với sức chứa và thiết bị khác nhau. Các hàm `createRoom`, `updateRoom`, `deleteRoom` kiểm tra tầng trong khoảng 1–8, sức chứa nguyên dương, tên không trùng và không cho xóa phòng còn yêu cầu đang hoạt động. Hàm `searchRooms` lọc theo từ khóa, tầng, sức chứa tối thiểu, thiết bị và loại khóa. `RoomSearchScreen` hiển thị kết quả trên sơ đồ tầng theo trạng thái khả dụng. `RoomManagementScreen` hợp nhất quản lý phòng và bảo trì: quản trị viên chọn tầng, chọn phòng rồi sửa, xóa, lên lịch bảo trì hoặc xem lịch sử sử dụng.

2.4.3 Module booking

Đây là module lớn nhất (`bookingRepository.ts`, khoảng 760 dòng), tập trung toàn bộ quy tắc đặt phòng. Hàm `validateBooking` ở Đoạn mã 2.2 kiểm tra lần lượt: phòng tồn tại và sẵn sàng, định dạng ngày giờ, cửa sổ đặt trước, khoảng thời gian chưa kết thúc, không trùng bảo trì, không trùng booking khác của phòng, không trùng lịch của chính người đặt, giới hạn số yêu cầu đang hoạt động và mục đích sử dụng không rỗng. Hai khoảng thời gian được coi là trùng khi cùng ngày và `start < other.end` và `end > other.start`.

Đoạn mã 2.2. Kiểm tra quy tắc khi tạo booking (rút gọn)

```
if (!room || room.status !== 'AVAILABLE') throw new Error('Phòng không tồn
↪ tại...');
if (dayDiff < cfg.minAdvanceDays || dayDiff > cfg.maxAdvanceDays)
  throw new Error(`Chỉ được đặt trước từ ${cfg.minAdvanceDays} đến ...`);
if (await hasMaintenanceConflict(roomId, date, startTime, endTime))
  throw new Error('Phòng có lịch bảo trì trong khoảng thời gian này.');
```

```
const active = bookings.filter(b => isBookingStillActive(b));
if (active.some(b => b.roomId === roomId && overlaps(b, date, startTime,
↪ endTime)))
  throw new Error('Phòng đã có yêu cầu khác trong khoảng thời gian này.');
```

```
if (active.filter(b => b.requesterUsername === user).length >=
↪ cfg.maxActiveBookingsPerUser)
  throw new Error('Mỗi người chỉ có tối đa ... yêu cầu');
```

```
if (!input.purpose.trim()) throw new Error('Vui lòng nhập mục đích sử dụng
↪ phòng.');
```

Chức năng đặt lặp hàng tuần sinh tối đa 8 lượt cách nhau 7 ngày, gán chung `recurringSeriesId` và chỉ áp dụng cửa sổ đặt trước, giới hạn số yêu cầu cho lượt đầu tiên. Các hàm `reviewRecurringSeries`, `updatePendingRecurringBookings` và `cancelFutureRecurringBookings` tuân theo nguyên tắc “tất cả hoặc không”: hệ thống kiểm tra toàn bộ chuỗi trước và chỉ ghi dữ liệu khi không lượt nào vi phạm. Khi duyệt, `reviewBooking` kiểm tra lại lịch đã duyệt và lịch bảo trì, sau đó cấp quyền tạo mã nếu phòng dùng khóa số. `changeBookingRoom` chỉ chấp nhận phòng có cùng loại khóa, sức chứa không nhỏ hơn và đủ thiết bị so với phòng cũ. `confirmBookingNoShow` chỉ ghi nhận vắng mặt sau thời gian ân hạn và khi chưa có bằng chứng check-in; hệ thống đồng thời giải phóng khung giờ và thu hồi mã.

2.4.4 Module schedule_maintenance

Module quản lý hai loại dữ liệu: lịch bảo trì (`MaintenanceRecord`) do quản trị viên tạo và yêu cầu bảo trì (`MaintenanceRequest`) do người dùng gửi. `createMaintenance` từ chối lịch trùng với bảo trì khác hoặc lịch đặt đã duyệt, yêu cầu quản trị viên đổi phòng trước khi tạo lịch bảo trì. `requestRoomMaintenance` chỉ cho phép báo sự cố trong thời gian sử dụng một lịch đặt đã duyệt của chính người gửi; mỗi lượt sử dụng có tối đa một yêu cầu chờ

xử lý. Phòng có yêu cầu đang chờ được đánh dấu trên sơ đồ quản trị.

2.4.5 Module access_control

Quyền tự tạo mã được lưu theo cặp người dùng-phòng (`RoomPinPermission`), không gắn cố định với toàn bộ hệ thống. Vì vậy, quản trị viên có thể thu hồi quyền tại một phòng mà không ảnh hưởng phòng khác. `createTemporaryPin` kiểm tra lịch đặt đã duyệt, loại khóa, mã còn hiệu lực, chủ lịch đặt và quyền tại phòng. Mã gồm 7 chữ số, có hiệu lực từ `start - pinGraceMinutes` đến `end + pinGraceMinutes`. Mã được lưu cục bộ trước; nếu phiên `OneIoT` đang mở thì gửi ngay, nếu chưa thì `deliverPendingTemporaryPins` gửi các mã còn chờ khi kết nối (Đoạn mã 2.3). Cách làm này cho phép ứng dụng hiển thị mã ngay cả khi tạm thời mất kết nối.

Đoạn mã 2.3. Lưu mã trước, gửi tới khóa khi có kết nối (rút gọn)

```
const code = generateSevenDigitPin();
const validFrom = addMinutes(start, -cfg.pinGraceMinutes);
const validUntil = addMinutes(end, cfg.pinGraceMinutes);
const { lock, passwordId } = await reserveSmartLockPasswordId(booking.roomId);
let updated = await saveTemporaryPin(bookingId, { code, validFrom, validUntil,
                                                lockPasswordId: passwordId, ...
                                                ↪ });
const status = await getOneIoTConnectionStatus().catch(() => ({connected:false}));
if (status.connected) updated = await deliverTemporaryPin(updated, lock);
```

2.4.6 Module smart_lock và tích hợp OneIoT

Ứng dụng giao tiếp với SmartLock thông qua nền tảng OneIoT bằng giao thức MQTT bảo mật. Hệ thống cho phép gửi mã PIN tạm thời tới khóa và nhận các sự kiện truy cập để cập nhật trạng thái booking. Thông tin xác thực OneIoT chỉ được lưu trong phiên làm việc và được xóa khi người dùng đăng xuất hoặc ứng dụng ngừng kết nối.

2.4.7 Module notifications, profile và configuration

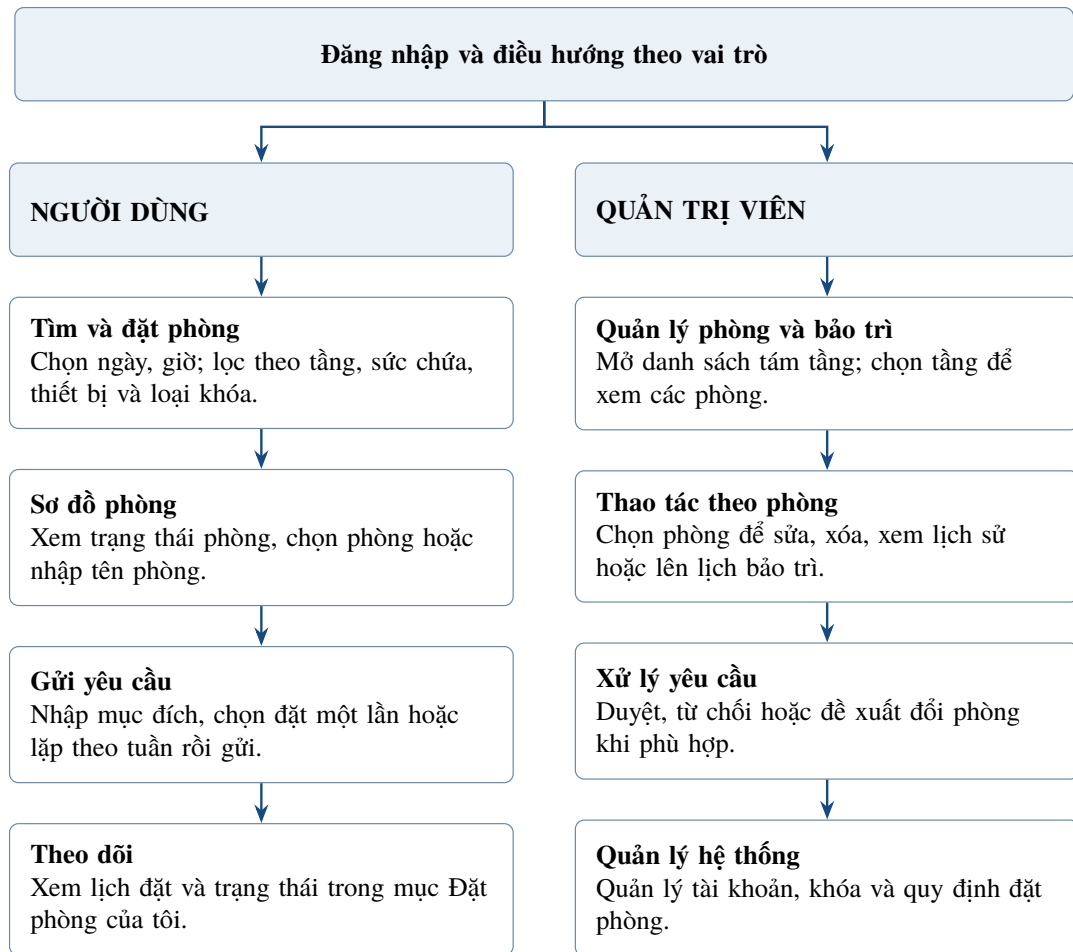
Hàm `addNotification` được gọi khi phát sinh sự kiện như tạo yêu cầu, duyệt, từ chối, cấp mã hoặc đổi phòng. Hàm `getNotificationTone` phân loại thông báo thành tích cực, tiêu cực, cần chú ý hoặc trung tính để sử dụng màu sắc nhất quán. Sự kiện ưu tiên được hiển thị bằng thông báo nổi thông qua module native `PriorityNotificationPackage`. Module `profile` lưu họ tên, email, số điện thoại và đơn vị theo tài khoản. Module `configuration` kiểm tra các tham số là số nguyên không âm, số ngày tối thiểu không lớn hơn tối đa và giới hạn yêu cầu ít nhất là 1 trước khi lưu.

2.4.8 Booking server

Booking server gồm các lớp chính: `ApiController` công khai API cho tài khoản, phòng, cấu hình, bảo trì, lịch đặt, quyền tạo mã và thông báo; `BookingService` thực thi quy tắc nghiệp vụ; `SessionAuthFilter` kiểm tra token Bearer; `ApiExceptionHandler` chuẩn hóa lỗi; `DataSeeder` tạo dữ liệu ban đầu. Mặc định server dùng H2 dạng tệp cục bộ; cấu hình `remote` cho phép kết nối cơ sở dữ liệu từ xa qua SSL. Mã nguồn đã sẵn sàng cho kiểm thử đồng bộ, trong khi bản trình diễn đặt `ENABLE_REMOTE_SYNC = false`.

2.5 Điều hướng giao diện

Hình 2.3 tóm tắt cấu trúc điều hướng được đối chiếu với mã nguồn hiện tại. Ở vai trò người dùng, thao tác tìm phòng và gửi yêu cầu nằm trong cùng màn hình `RoomSearchScreen`: sau khi chọn khung giờ và phòng, người dùng nhập mục đích rồi gửi yêu cầu trực tiếp. Ở vai trò quản trị viên, `RoomManagementScreen` gộp thông tin phòng và bảo trì; quản trị viên chọn tầng, chọn phòng rồi thực hiện thao tác tương ứng. Các chức năng khác được mở từ màn hình chính theo vai trò. Ảnh chụp giao diện ứng dụng hiện tại được trình bày tại Phụ lục C.



Hình 2.3. Luồng điều hướng giao diện theo hai vai trò

CHƯƠNG 3

KIỂM THỬ VÀ ĐÁNH GIÁ

3.1 Mục tiêu và phạm vi kiểm thử

Hoạt động kiểm thử tập trung vào ba mục tiêu: xác nhận quy tắc nghiệp vụ, kiểm tra khả năng hoạt động độc lập của bản Release và đánh giá luồng đồng bộ khi có máy chủ. Mỗi thay đổi chỉ được chấp nhận khi vượt qua kiểm tra mã nguồn, kiểm thử tự động và kiểm tra khởi động trên thiết bị thật. Cách tiếp cận này giúp tách lỗi nghiệp vụ khỏi lỗi môi trường hoặc lỗi kết nối [6].

Phạm vi kiểm thử bao gồm tài khoản, tìm và đặt phòng, phê duyệt, đặt laptop, bảo trì, quyền tạo PIN, thông báo, cấu hình và đồng bộ dữ liệu. Việc xác nhận mã PIN trên khóa vật lý và vận hành với máy chủ triển khai diện rộng nằm ngoài lượt kiểm thử được ghi nhận trong báo cáo này.

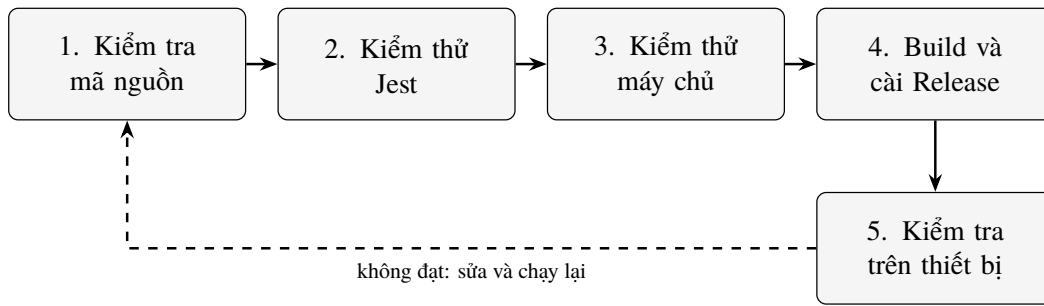
3.2 Môi trường kiểm thử

Bảng 3.1. Môi trường kiểm thử của phiên bản báo cáo

Hạng mục	Cấu hình và mục đích
Ứng dụng di động	React Native/TypeScript; dữ liệu cục bộ bằng Async Storage; <code>ENABLE_REMOTE_SYNC=false</code> cho bản cài trên điện thoại.
Thiết bị	Điện thoại Android thật, cài trực tiếp bản APK Release.
Kiểm thử tự động	Jest cho lớp nghiệp vụ và giao diện; TypeScript kiểm tra kiểu; ESLint kiểm tra quy ước mã nguồn.
Máy chủ kiểm thử	Spring Boot khởi động trên cổng ngẫu nhiên, cơ sở dữ liệu H2 trong bộ nhớ; không phụ thuộc máy chủ đang triển khai.
Theo dõi thiết bị	ADB, UI Automator và logcat để xác nhận màn hình, tiến trình và lỗi nghiêm trọng.
Tài khoản trình diễn	Hai tài khoản thử nghiệm tương ứng với vai trò quản trị viên và người dùng.

3.3 Quy trình kiểm thử

Quy trình kiểm thử được thực hiện theo năm bước ở Hình 3.1. Khi một bước không đạt, nhóm sửa nguyên nhân tại module liên quan và chạy lại từ bước kiểm tra mã nguồn. Cách tổ chức này ngăn việc tạo APK từ phiên bản chưa vượt qua kiểm thử hồi quy.



Hình 3.1. Quy trình kiểm thử và xác nhận bản Release

1. Kiểm tra kiểu dữ liệu và quy ước mã nguồn bằng TypeScript và ESLint.
2. Chạy toàn bộ bộ kiểm thử Jest ở chế độ tuần tự.
3. Khởi động Spring Boot với H2 trong bộ nhớ và chạy kiểm thử tích hợp REST API.
4. Build APK Release, cài đề lên thiết bị và khởi động ứng dụng.
5. Đăng nhập, kiểm tra màn hình theo vai trò và đọc logcat để phát hiện lỗi nghiêm trọng.

3.4 Nhóm ca kiểm thử chính

Bảng 3.2 tóm tắt các nhóm ca kiểm thử được duy trì trong mã nguồn. Các ca được tổ chức theo nghiệp vụ thay vì phụ thuộc vào thứ tự thao tác trên giao diện, nhờ đó có thể chạy lặp lại sau mỗi lần hợp nhất nhánh.

Bảng 3.2. Các nhóm ca kiểm thử chính

Mã	Phạm vi	Nội dung xác nhận
TC-01	Tài khoản và vai trò	Khởi tạo hai tài khoản trình diễn; đăng nhập đúng/sai; đăng ký; đổi và khôi phục mật khẩu; chặn tài khoản bị vô hiệu hóa.
TC-02	Phân loại thông báo	Thông báo phê duyệt, cấp mã và check-in dùng trạng thái tích cực; từ chối, hủy và lỗi dùng trạng thái tiêu cực; hủy hiệu chỉ đếm yêu cầu còn hiệu lực.
TC-03	Tìm và quản lý phòng	Lọc theo tầng, thời gian, sức chứa và thiết bị; quản lý phòng theo từng tầng; kết hợp thông tin phòng và bảo trì trên cùng màn hình.

Mã	Phạm vi	Nội dung xác nhận
TC-04	Quy tắc đặt phòng	Kiểm tra cửa sổ đặt trước, thời gian đã kết thúc, trùng phòng, trùng lịch người dùng, bảo trì và giới hạn số yêu cầu đang hoạt động.
TC-05	Phê duyệt và đổi phòng	Duyệt/từ chối; cấp quyền tạo mã theo phòng; chỉ đề xuất phòng thay thế có cùng loại khóa, đủ sức chứa và thiết bị.
TC-06	Đặt lặp hàng tuần	Tạo chuỗi đặt phòng, duyệt cả chuỗi, sửa hoặc hủy các lượt tương lai và giữ dữ liệu nhất quán.
TC-07	Bảo trì và sự cố	Người dùng chỉ được báo sự cố trong phiên đã duyệt; quản trị viên nhận yêu cầu và phòng liên quan được đánh dấu để xử lý.
TC-08	PIN tạm thời	Sinh đúng 7 chữ số, tính thời hạn theo lịch đặt, lưu trước khi gửi và duy trì trạng thái chờ khi chưa có phiên OneIoT.
TC-09	Đồng bộ REST API	Đăng nhập, cấu hình, hồ sơ, lịch đặt lặp, phê duyệt, gắn khóa, lưu PIN và cô lập dữ liệu theo tài khoản trên Spring Boot/H2.
TC-10	Bản Release trên thiết bị	Cài APK, mở ứng dụng không cần Metro, đăng nhập tài khoản thử nghiệm, hiển thị màn hình theo vai trò và không phát sinh lỗi nghiêm trọng.

3.5 Kết quả

Kết quả gần nhất được tổng hợp tại Bảng 3.3. Các lệnh được chạy trên cùng mã nguồn dùng để tạo PDF này và trước khi hợp nhất nội dung báo cáo vào nhánh chính.

Bảng 3.3. Kết quả kiểm thử gần nhất

Hạng mục	Kết quả	Bằng chứng
TypeScript	Đạt	<code>npm run tsc --noEmit</code> kết thúc với mã 0.
ESLint	Đạt	<code>npm run lint</code> không phát hiện lỗi.
Jest	Đạt	6 bộ kiểm thử, 36/36 ca đạt.
Spring Boot/H2	Đạt	<code>RemoteSyncIntegrationTest</code> : 1/1 ca tích hợp đạt.
Build Release	Đạt	Gradle hoàn thành và tạo <code>app-release.apk</code> .
Thiết bị Android	Đạt	Cài đặt thành công; ứng dụng mở độc lập; đăng nhập tài khoản thử nghiệm; không có lỗi nghiêm trọng trong logcat.

3.6 Giới hạn và hướng kiểm chứng tiếp theo

Bộ kiểm thử hiện tại chứng minh luồng cục bộ hoạt động ổn định và lớp REST API có thể đồng bộ với máy chủ Spring Boot/H2 trong môi trường kiểm thử. Kết quả này chưa thay thế kiểm thử tải, kiểm thử đồng thời trên nhiều điện thoại hoặc kiểm thử với máy chủ triển khai thật. Bước tiếp theo là chạy cùng kịch bản trên ít nhất hai thiết bị dùng chung máy chủ, mô phỏng mất kết nối giữa giao dịch và xác nhận dữ liệu sau khi kết nối lại.

Đối với OneIoT, ứng dụng đã có cơ chế nhập token theo phiên, kiểm tra kết nối, xếp hàng lệnh tạo PIN và ngắt phiên khi đóng ứng dụng. Việc xác nhận khóa vật lý nhận lệnh, áp dụng đúng thời hạn và phát sự kiện check-in/check-out cần được thực hiện khi thiết bị khóa thật sẵn sàng.

KẾT LUẬN

Phiên bản giữa kỳ của RoomHub đã hình thành một ứng dụng Android có thể trình diễn trọn vẹn luồng cục bộ trên một thiết bị. Người dùng có thể tìm và đặt phòng, theo dõi lịch, xử lý quyền truy cập và báo sự cố; quản trị viên có thể quản lý tài khoản, phòng, bảo trì, yêu cầu đặt phòng, cấu hình nghiệp vụ và khóa thông minh. Mã nguồn được chia thành mười module nghiệp vụ, tách phần giao diện, dịch vụ và mô hình dữ liệu để thuận lợi cho phát triển theo nhóm.

Bộ kiểm thử hiện tại gồm 36 ca Jest, kiểm tra TypeScript, ESLint và một ca tích hợp Spring Boot/H2 cho các luồng đồng bộ quan trọng. Bản Release đã được build, cài và đăng nhập thành công trên điện thoại Android thật ở chế độ cục bộ, không phụ thuộc Metro hoặc máy chủ. Đây là cấu hình phù hợp cho buổi trình diễn khi hạ tầng trực tuyến chưa được bảo đảm.

Trong giai đoạn tiếp theo, nhóm sẽ kiểm thử đồng bộ trên nhiều thiết bị với máy chủ triển khai thật, bổ sung kịch bản mất kết nối và xung đột cập nhật, đồng thời xác nhận đầu cuối với SmartLock vật lý. Các chức năng mở rộng như thống kê mức sử dụng phòng, gợi ý phòng và phương án check-in bổ sung sẽ được xem xét sau khi ba luồng trên ổn định.

TÀI LIỆU THAM KHẢO

- [1] Meta Platforms, “React Native Documentation.” <https://reactnative.dev/docs/getting-started>. Truy cập 10/2026.
- [2] React Native Community, “Async Storage Documentation.” <https://react-native-async-storage.github.io/async-storage/>. Truy cập 10/2026.
- [3] Broadcom, “Spring Boot Reference Documentation.” <https://docs.spring.io/spring-boot/>. Truy cập 10/2026.
- [4] OASIS, “MQTT Version 3.1.1, OASIS Standard,” 2014. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>.
- [5] oneM2M Partners, “oneM2M TS-0010: MQTT Protocol Binding.” <https://www.onem2m.org/technical/published-specifications>. Truy cập 10/2026.
- [6] ISTQB, “Certified Tester Foundation Level Syllabus v4.0,” 2023. <https://www.istqb.org/>.
- [7] OpenJS Foundation, “Jest Documentation.” <https://jestjs.io/docs/getting-started>. Truy cập 10/2026.

PHỤ LỤC A

THÀNH PHẦN VÀ CẤU TRÚC DỰ ÁN

A.1 Cấu trúc kho mã nguồn

Kho mã nguồn <https://github.com/xwomen1/classroom-booking> được tổ chức như Đoạn mã A.1. Kho chỉ lưu mã nguồn cần thiết; tệp APK, bộ nhớ đệm, node_modules và thư mục build không được đưa lên Git, mỗi thành viên tự build sau khi clone.

Đoạn mã A.1. Cấu trúc thư mục chính

```
classroom-booking/
|-- README.md                Hướng dẫn clone, build, cài APK
|-- du_an_dat_phong.tex/.pdf  Đề xuất dự án
|-- workflow_booking_app.tex/.pdf Workflow nghiệp vụ
|-- hien_trang_trien_khai.tex/.pdf Hiện trạng triển khai
|-- code/
|   |-- booking_classroom/    Ứng dụng React Native (Android)
|   |   |-- App.tsx           Điểm vào, điều hướng theo vai trò
|   |   |-- src/core/         API client, tham số cấu hình, lưu trữ
|   |   |-- src/shared/       component dùng chung
|   |   |-- src/modules/      10 module nghiệp vụ
|   |   |-- __tests__/        36 ca kiểm thử Jest
|   |   |-- android/.../OneIoTMqttPackage.kt  MQTT/TLS tới OneIoT
|   |   |-- android/.../PriorityNotificationPackage.kt  thông báo nổi
|   |   |-- docs/, evidence/  sơ đồ luồng, ảnh kiểm tra thiết bị
|   |   |-- scripts/          build Debug/Release (PowerShell)
|   |-- booking-server/       Spring Boot + H2
|       |-- src/main/java/com/bookingclassroom/server/
|           ApiController, BookingService, SessionAuthFilter,
|           ApiExceptionHandler, DataSeeder, Domain
|-- tunghv2/                  Giả lập SmartLock (Python)
|-- tunghv3/                  Công cụ giao tiếp OneIoT (Python)
```

A.2 Build và cài đặt

Ứng dụng được build và cài đặt bằng các lệnh trong Đoạn mã A.2. Bản Release chạy độc lập sau khi cài, không cần Metro, cáp USB hay booking server.

Đoạn mã A.2. Build và cài đặt bản Release

```
git clone https://github.com/xwomen1/classroom-booking.git
cd classroom-booking/code/booking_classroom
npm ci
npm run build:android:release
adb install -r ./android/app/build/outputs/apk/release/app-release.apk
npm test      # chạy bộ kiểm thử Jest
npm run lint   # kiểm tra ESLint
```

A.3 Tổng hợp tài liệu và phụ lục

Bảng A.1 tóm tắt các tài liệu hỗ trợ. Tên tệp và thư mục tương ứng được liệt kê ở Đoạn mã A.1; nhật ký kiểm tra thiết bị nằm trong thư mục `code/booking_classroom/evidence/`.

Bảng A.1. Tài liệu hỗ trợ cho báo cáo

Tài liệu	Nội dung
Đề xuất dự án	Phạm vi và tiêu chí kiểm chứng
Quy trình nghiệp vụ	Đặt phòng, phê duyệt và bàn giao
Hiện trạng triển khai	Các chức năng đã kiểm tra
Nhật ký kiểm thử	Kết quả cài và mở bản Release trên thiết bị Android
Phụ lục B	Prompt sử dụng trong quá trình phát triển

A.4 Phân công nhóm

Bảng A.2 tổng hợp phần việc chính của các thành viên.

Bảng A.2. Phân công công việc trong nhóm

Thành viên	Phần việc chính
Hoàng Văn Tùng	Kiến trúc và giao diện ứng dụng; nghiệp vụ cục bộ; tích hợp OneIoT và SmartLock; build Release.
Đỗ Thùy Linh	Booking Server với Spring Boot/H2; thông báo ưu tiên; logo và cải tiến màn hình đăng nhập.
Nguyễn Đình Phúc	Đặt phòng lặp, quản lý chuỗi và no-show; kiểm thử tự động cho chức năng đặt phòng.
Nguyễn Việt Dũng	Kịch bản và kiểm thử thủ công; tổng hợp vấn đề; hỗ trợ tài liệu dự án.

PHỤ LỤC B

PROMPT HỖ TRỢ QUÁ TRÌNH PHÁT TRIỂN

Nhóm sử dụng trợ lý lập trình để hỗ trợ phân tích yêu cầu, rà soát mã, đề xuất kiểm thử và soạn tài liệu. Trợ lý không tự quyết định phạm vi sản phẩm hoặc xác nhận một chức năng đã hoàn thành. Mọi thay đổi đều phải được thành viên phụ trách đọc lại, chạy kiểm thử và kiểm tra trên nhánh làm việc trước khi hợp nhất.

B.1 Nguyên tắc sử dụng

1. Nêu rõ mục tiêu, module, tệp liên quan và quy tắc nghiệp vụ cần giữ.
2. Giới hạn phạm vi thay đổi để tránh tác động ngoài ý muốn tới module khác.
3. Yêu cầu chỉ ra giả định và đối chiếu với mã nguồn trước khi sửa.
4. Luôn kèm bước kiểm tra phù hợp như Jest, TypeScript, ESLint, kiểm thử tích hợp hoặc build APK.
5. Không đưa token, mật khẩu thật hoặc dữ liệu nhạy cảm vào mã nguồn và lịch sử Git.
6. Chỉ commit sau khi xem lại diff; chỉ push khi người phụ trách cho phép.

B.2 Các nhóm prompt tiêu biểu

Bảng B.1. Prompt tiêu biểu trong quá trình phát triển

Mục đích	Nội dung rút gọn
Phân tích nghiệp vụ	“Đối chiếu yêu cầu đặt phòng với mã hiện tại; xác định thực thể, trạng thái và quy tắc còn thiếu. Không tự bổ sung yêu cầu chưa có căn cứ.”
Tổ chức module	“Sắp xếp mã theo module để các thành viên phát triển độc lập; chỉ rõ màn hình, dịch vụ, mô hình dữ liệu và API công khai của từng module.”
Triển khai chức năng	“Cài đặt luồng tạo PIN tạm thời: mã gồm 7 chữ số, thời hạn lấy từ lịch đặt, lưu cục bộ trước và chỉ gửi khi phiên OneIoT đã kết nối.”
Rà soát đồng bộ	“So sánh repository cục bộ với REST API; tìm sai khác về quyền, trạng thái và cấu trúc dữ liệu; kiểm thử bằng Spring Boot/H2 cục bộ khi máy chủ thật chưa hoạt động.”

Mục đích	Nội dung rút gọn
Kiểm thử	“Viết hoặc cập nhật ca kiểm thử cho quy tắc vừa sửa; chạy Jest, TypeScript và ESLint; báo rõ lệnh, kết quả và giới hạn chưa kiểm chứng.”
Hợp nhất nhánh	“Đọc toàn bộ xung đột; ưu tiên thay đổi mới của nhánh được hợp nhất, nhưng giữ chức năng hiện có nếu nhánh đó làm mất endpoint hoặc quy tắc đã triển khai.”
Build thiết bị	“Build APK Release với chế độ cục bộ, cài lên điện thoại thật, mở ứng dụng, đăng nhập tài khoản trình diễn và kiểm tra log lỗi nghiêm trọng.”
Rà soát tài liệu	“Đối chiếu báo cáo với mã nguồn và kết quả kiểm thử; loại bỏ khẳng định không có bằng chứng; chuẩn hóa thuật ngữ, bảng biểu, hình và tham chiếu.”

B.3 Quy trình kiểm soát đầu ra

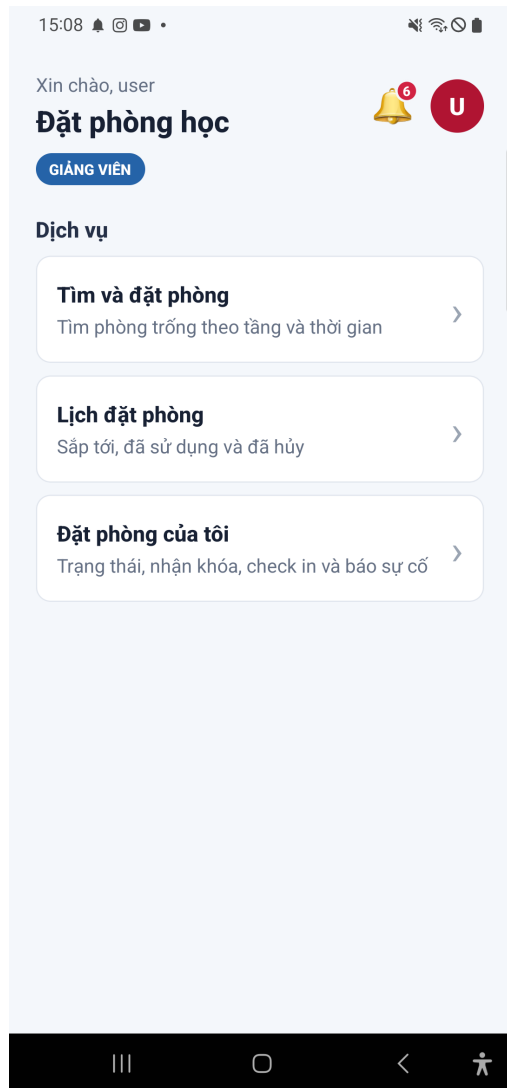
Đầu ra từ trợ lý được kiểm soát qua bốn bước: (1) xem danh sách tệp thay đổi; (2) đọc diff và đối chiếu quy tắc nghiệp vụ; (3) chạy bộ kiểm tra tương ứng; (4) kiểm tra thủ công luồng chính trên bản Release nếu thay đổi ảnh hưởng giao diện hoặc tích hợp thiết bị. Nếu một bước không đạt, thay đổi được sửa tại nhánh làm việc và kiểm tra lại trước khi commit.

Các nội dung chưa thể kiểm chứng, chẳng hạn máy chủ thật đang tắt hoặc khóa vật lý chưa sẵn sàng, phải được ghi rõ là giới hạn. Báo cáo không coi kết quả mô phỏng là bằng chứng cho hoạt động của hạ tầng thật.

PHỤ LỤC C

ẢNH GIAO DIỆN ỨNG DỤNG HIỆN TẠI

Các ảnh dưới đây được chụp trực tiếp từ phiên bản ứng dụng hiện tại cài trên điện thoại Android. Giao diện theo vai trò, thao tác tìm và đặt phòng trên cùng một màn hình, cùng cách quản lý phòng theo tầng khớp với luồng điều hướng ở Hình 2.3.



(a) Màn hình người dùng



(b) Màn hình quản trị viên

Hình C.1. Màn hình chính theo hai vai trò của ứng dụng hiện tại

18:51





Tìm và đặt phòng

Nhập nhanh tên phòng

Tên phòng

A101

Khoảng thời gian

Vuốt dọc từng cột rồi chạm để chọn

Ngày

Bắt đầu

Kết thúc

05/10

07:00

08:30

06/10

07:30

09:00

07/10

08:00

09:30

2026-10-06 · 08:00–09:00

Bộ lọc phòng

Sức chứa tối thiểu

Thiết bị cần có

Mọi khóa

Mã số

Cơ/thẻ

☐ Trống

☐ Đã đặt

☐ Bảo trì

☐ Không khớp lọc

Đã chọn phòng A101.

TÀA A

T8

T7

T6

T5

T4

TẦNG 1

A101

30 chỗ

Máy chiếu, Điều hòa

A102

45 chỗ

TV, Bảng thông minh

A103

60 chỗ

Máy chiếu, Micro

A104

35 chỗ

Điều hòa, Camera

(a) Chọn thời gian và lọc phòng

18:51



<

Tìm và đặt phòng

TÒA A

T8

T7

T6

T5

T4

T3

T2

T1

TẦNG 1

A101

30 chỗ

Máy chiếu, Điều hòa

A102

45 chỗ

TV, Bảng thông minh

A103

60 chỗ

Máy chiếu, Micro

A104

35 chỗ

Điều hòa, Camera

A105

50 chỗ

Máy chiếu, Loa

A106

40 chỗ

TV, Điều hòa

A107

80 chỗ

Máy chiếu, Micro, Camera

A101 · Đang trống

30 chỗ · Máy chiếu, Điều hòa

Khóa mã số online

Mục đích sử dụng

Ví dụ: Họp nhóm đồ án

Tần suất đặt

Chọn lịch một lần hoặc lặp hàng tuần

Một lần

Hàng tuần

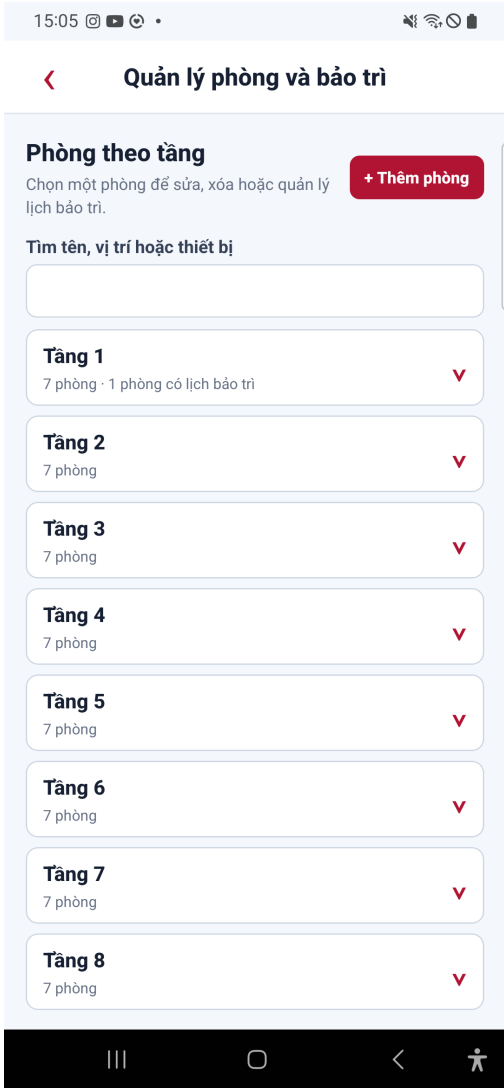
Đặt phòng A101

(b) Chọn phòng và nhập thông tin đặt

Hình C.2. Tìm phòng và tạo yêu cầu trên cùng một màn hình

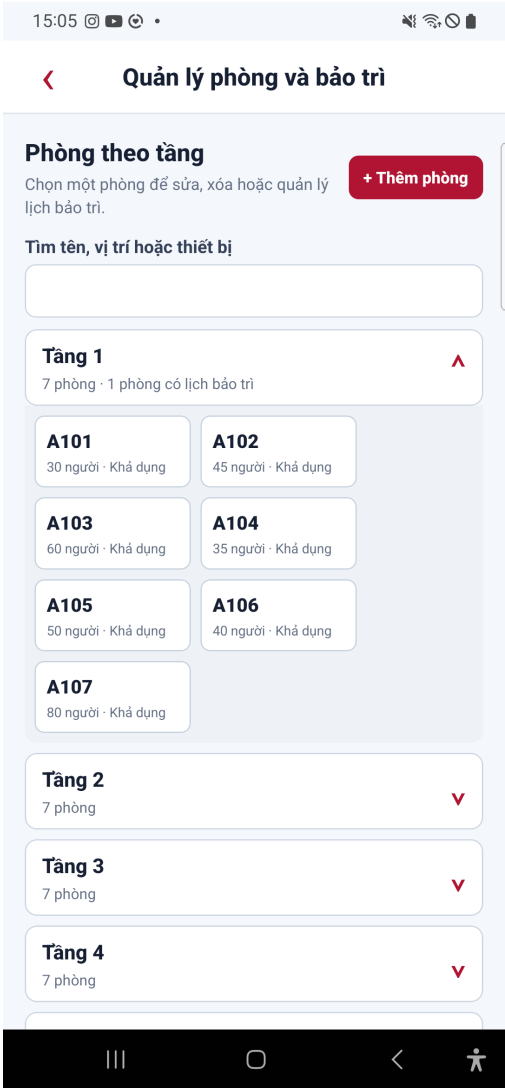


(a) Sơ đồ phòng theo tầng



(b) Danh sách tầng của quản trị viên

Hình C.3. Hai cách xem phòng theo tầng ở ứng dụng hiện tại



(a) Phòng thuộc tầng 1



(b) Cấu hình quy tắc đặt phòng

Hình C.4. Giao diện quản lý phòng và cấu hình nghiệp vụ